

Aula 10 - Validação e Tratamento de Erros no Spring Boot

Duração: 4 horas (1h teórica + 3h prática)

Parte Teórica (1 hora)

1. Revisão da Aula Anterior (15 min)

- Discussão sobre:
 - Operações CRUD com Spring Data JPA.
 - Dúvidas sobre consultas customizadas.

2. Validação de Dados (25 min)

Anotações de Validação (Jakarta Validation)

Anotação	Descrição
@NotBlank	String não nula e não vazia
@Size(min, max)	Tamanho de string/coleção
@Positive	Número estritamente positivo
@Email	Formato de e-mail válido
@Pattern(regex)	Regex customizado

Habilitando Validação

```
java

@PostMapping
public ResponseEntity<?> create(@Valid @RequestBody ProductDTO dto) {
    // Lógica
}
```

3. Tratamento de Erros (20 min)

Estratégias Comuns

- `ResponseEntityExceptionHandler` : Classe base para tratamento global.
- `@ExceptionHandler` : Métodos para erros específicos.
- **DTOs de Erro Padronizados:**

```
java
public record ErrorResponse(String message, LocalDateTime timestamp) {}
```

HTTP Status Codes

Código	Quando Usar
400 (Bad Request)	Validação falhou
404 (Not Found)	Recurso não encontrado
500 (Internal Error)	Erro inesperado no servidor

Parte Prática (3 horas)

Atividade: Sistema de Validação e Tratamento de Erros

Passo 1: Implementar Validações (1h)

1. DTO de Criação (`ProductRequestDTO`):

```
java
public record ProductRequestDTO(
    @NotBlank(message = "Nome é obrigatório")
    @Size(min = 3, max = 100, message = "Nome deve ter 3-100 caracteres")
    String name,

    @Positive(message = "Preço deve ser positivo")
    double price
) {}
```

2. Atualizar o Controller:

```
java
```

```

@PostMapping
public ResponseEntity<Product> create(@Valid @RequestBody ProductRequestDTO dto) {
    Product product = new Product(dto.name(), dto.price());
    return ResponseEntity.ok(service.create(product));
}

```

3. Teste com Postman:

- o Envie um JSON sem `name` e veja o erro 400 automático.

Passo 2: Tratamento Global de Erros (1h)

1. Classe `GlobalExceptionHandler`:

```

java

@RestControllerAdvice
public class GlobalExceptionHandler extends ResponseEntityExceptionHandler {

    @ExceptionHandler(EntityNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(EntityNotFoundException ex) {
        return ResponseEntity
            .status(HttpStatus.NOT_FOUND)
            .body(new ErrorResponse(ex.getMessage(), LocalDateTime.now()));
    }

    @Override
    protected ResponseEntity<Object> handleMethodArgumentNotValid(
        MethodArgumentNotValidException ex,
        HttpHeaders headers,
        HttpStatusCode status,
        WebRequest request
    ) {
        Map<String, String> errors = ex.getFieldErrors().stream()
            .collect(Collectors.toMap(
                FieldError::getField,
                FieldError::getDefaultMessage
            ));
        return ResponseEntity.badRequest().body(errors);
    }
}

```

2. Lançar exceções no Service:

```

java

public Product findById(Long id) {
    return repository.findById(id)
        .orElseThrow(() -> new EntityNotFoundException("Produto não encontrado"));
}

```

Passo 3: Testes Avançados (1h)

1. Cenários para testar:

- POST com nome vazio → Erro 400 com detalhes.
- GET com ID inválido → Erro 404 personalizado.
- DELETE com ID inexistente → Erro 404.

2. Melhorando o DTO de Erro:

```
java

public record ErrorResponse(
    String message,
    LocalDateTime timestamp,
    Map<String, String> details // Para erros de validação
) {}
```

Tarefa para Casa

1. Validação Customizada:

- Criar uma anotação `@ValidProductName` para evitar nomes inválidos (ex: "teste").

2. Tratar `DataIntegrityViolationException`:

- Retornar HTTP 409 (Conflict) para duplicatas.

Próxima Aula

- **Tópico:** Testes Unitários com JUnit 5 e Mockito.
- **Atividade prática:** Testar controllers e services.

Resultado Esperado

- API com:
 - Validações declarativas nos DTOs.
 - Erros padronizados e fáceis de debugar.
 - Respostas HTTP semânticas.

- ✓ Validações funcionais
- ✓ Tratamento global de erros
- ✓ Pronto para implementar testes

<https://www.baeldung.com/wp-content/uploads/2017/10/spring-rest-validation-flow.png> (*Fluxo de validação no Spring*) 🚀